Advanced Math

Matrix exponentiation, FFT/NTT, Gaussian elimination & XOR bases

Learning goals

- Model "n steps of a linear process" as a **matrix power**, including semiring variants
- Build companion matrices for linear recurrences (with constants / polynomial terms)
- Explain FFT as evaluate → multiply pointwise → interpolate, and implement NTT
- Recognize hidden convolutions: counting pairs, wildcard matching, sumsets
- Solve linear systems mod p / over reals / over GF(2)
- Use the **XOR basis** for max-xor, k-th xor, and xor-path problems

Prereqs: Year weeks 11 (mod arithmetic), 30 (combinatorics); Regional 07

Motivating problems

1. Count sequences of dice throws summing to n, $n \le 10^{18}$ (CSES 1096)
2. Count shifts of pattern P in text T where every pattern letter has an equal text letter within distance k of its aligned position — $|T|, |P| \le 2 \cdot 10^5$ (CF 528D *Fuzzy Search*)
3. Max xor of a subset of a range a[l..r], $5 \cdot 10^5$ queries (CF 1100F *Ivan and Burgers*)

Three different "linear algebra" engines. Today: one lecture each.

Matrix exponentiation

If state vector `v_{t+1} = M · v_t`, then `v_n = Mⁿ · v_0`.

- Square-and-multiply: $O(k^3 \log n)$ for a $k \times k$ matrix
- Works over any **semiring** with associative $\otimes$ distributing over $\oplus$:
  - $(+, \times)$ mod p: counting walks of length n (CSES 1723 *Graph Paths I*)
  - $(\min, +)$: shortest walk with exactly n edges (CSES 1724 *Graph Paths II*)
  - (or, and): reachability in exactly n steps — use `bitset` rows

Associativity of the product is what makes repeated squaring valid.

Companion matrix

$a_i = c_1 a_{i-1} + \ldots + c_k a_{i-k}$

$$
\begin{pmatrix} a_i \\ a_{i-1} \\ \vdots \\ a_{i-k+1} \end{pmatrix} = \begin{pmatrix} c_1 & c_2 & \ldots & c_k \\ 1 & 0 & \ldots & 0 \\ 0 & 1 & \ldots & 0 \\ 0 & 0 & \ldots & 0 \end{pmatrix} \cdot \begin{pmatrix} a_{i-1} \\ a_{i-2} \\ \vdots \\ a_{i-k} \end{pmatrix}
$$

Extra terms → extra state:

- $+ d$ constant: add a coordinate that is always 1
- $+ i^2$: add coordinates $i^2, i, 1$ (binomial expansion of $(i+1)^2$)
- prefix sums $S_i = S_{i-1} + a_i$: add coordinate S

Matrix power code

```
using Mat = vector<vector<long long>>;
Mat mul(const Mat& A, const Mat& B) {
    int n = A.size(), m = B[0].size(), k = B.size();
    Mat C(n, vector<long long>(m, 0));
    for (int i = 0; i < n; i++)
        for (int t = 0; t < k; t++) {
            if (A[i][t] == 0) continue;
            for (int j = 0; j < m; j++) C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD;
        }
    return C;
}
Mat power(Mat A, long long e) {
    int n = A.size(); Mat R(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++) R[i][i] = 1;
    for (; e > 0; e >>= 1, A = mul(A, A)) if (e & 1) R = mul(R, A);
    return R;
}
```

i-t-j loop order is cache friendly. For many queries with the same matrix: precompute `M^(2^j)` and multiply **vectors** ($O(k^2 \log n)$ per query).

When k is large

- k ≤ ~100: matrices fine ($k^3 \log n \approx 6 \cdot 10^7$)
- k ~ 1000–3000: **Kitamasa** / polynomial `x^n mod charpoly` → $O(k^2 \log n)$
- With NTT: $O(k \log k \log n)$
- Unknown recurrence but you can compute first 2k terms? **Berlekamp–Massey**
  (World Finals track, Lecture 2)

Real example: CF 1182E *Product Oriented Recurrence* — take exponents of c, f1, f2, f3
separately; each obeys a tribonacci-like recurrence, computed **mod p − 1** (Fermat).

Polynomial multiplication = convolution

`C(x) = A(x)·B(x)`, `c_k = Σ_{i+j=k} a_i b_j`. Naive O(nm).

FFT plan:

1. **Evaluate** A and B at the N-th roots of unity, N ≥ n + m − 1 a power of 2
2. **Multiply** values pointwise — O(N)
3. **Interpolate** back (inverse DFT = DFT with ω^{-1}, divide by N)

Evaluation in O(N log N) by splitting even/odd coefficients: `A(x) = A_even(x²) + x·A_odd(x²)`, and squares of N-th roots are the (N/2)-th roots.

The butterfly

```
ω = e^{2πi/len}
for each block of size len, j < len/2:
    u = a[j], v = ω^j · a[j + len/2]
    a[j]           = u + v        // A(ω^j)
    a[j + len/2]   = u − v        // A(ω^{j + len/2}) since ω^{len/2} = −1
```

Iterative version: bit-reverse the input once, then len = 2, 4, …, N.

Correctness of inverse: $\Sigma_k \omega^{jk} = N \cdot [j \equiv 0 \bmod N]$ (geometric series).

NTT: FFT in a finite field

Need a primitive N-th root of unity mod p: requires **N | p − 1**.

prime	form	max N	root
998244353	$119 \cdot 2^{23}+1$	2^{23}	3
469762049	$7 \cdot 2^{26}+1$	2^{26}	3
167772161	$5 \cdot 2^{25}+1$	2^{25}	3

Exact, no precision issues — but only gives answers **mod p**.
Arbitrary modulus (e.g. 10^9+7): three NTT primes + CRT, or split-FFT with 15-bit halves.

NTT code

```
void ntt(vector<int>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        long long w = power(G, (MOD - 1) / len);
        if (invert) w = power(w, MOD - 2);
        for (int i = 0; i < n; i += len) {
            long long wn = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i + j], v = a[i + j + len / 2] * wn % MOD;
                a[i + j] = u + v < MOD ? u + v : u + v - MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
                wn = wn * w % MOD;
            }
        }
    }
    if (invert) { long long inv = power(n, MOD - 2); for (int& x : a) x = x * inv % MOD; }
}
```

Complex FFT: precision rules

- `double` FFT is exact when result coefficients $\lesssim 10^{14}$–10^{15} and N $\lesssim 10^{6}$
- Compute twiddles with `polar(1.0, ang*j)` (or a precomputed table), not by repeated multiplication — error accumulates
- Round with `llround`; never truncate
- Use NTT when the problem says "mod 998244353"; use FFT for exact counts of
  moderate size (e.g. 0/1 indicator vectors, values ≤ 10^{6})

Hidden convolutions

Problem shape	Convolution
# pairs (i, j) with a_i + b_j = s, for all s	counts × counts
All pairwise sums of a small-value set (sumset)	indicator × indicator
Pattern match with mismatches / wildcards	$\Sigma\ (p-t)^2\ p\ t$ expanded into 3 convolutions
Match count at every shift (per letter)	reverse pattern, convolve
Ways to pick k items from groups	product of many polys (D&C + NTT)

CF 528D: per letter, mark text positions within distance k of that letter; convolve with reversed pattern indicator; shift is good if all 4 letters fully match.

Gaussian elimination mod p

```cpp
// a: n x (m+1) augmented, p prime. returns rank or -1; x = one solution
int gaussMod(vector<vector<long long>> a, long long p, vector<long long>& x) {
    int n = a.size(), m = a[0].size() - 1, row = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && row < n; col++) {
        int sel = -1;
        for (int i = row; i < n; i++) if (a[i][col] % p != 0) { sel = i; break; }
        if (sel == -1) continue;
        swap(a[sel], a[row]); where[col] = row;
        long long inv = power(a[row][col], p - 2, p);
        for (int j = col; j <= m; j++) a[row][j] = a[row][j] * inv % p;
        for (int i = 0; i < n; i++) if (i != row && a[i][col]) {
            long long f = a[i][col];
            for (int j = col; j <= m; j++) a[i][j] = ((a[i][j] - f * a[row][j]) % p + p) % p;
        }
        row++;
    }
    for (int i = row; i < n; i++) if (a[i][m] % p) return -1;
    x.assign(m, 0);
    for (int c = 0; c < m; c++) if (where[c] != -1) x[c] = a[where[c]][m];
    return row;
}
```

#solutions = p^(m − rank), O(n·m·min(n,m))

Gaussian elimination: three fields

Field	Pivot rule	Typical use
reals	**largest |a|** (partial pivoting), compare with eps	expected values on cyclic Markov chains (CF 24D)
GF(p)	any nonzero	counting solutions, interpolation (CF 1155E)
GF(2)	any 1; rows as `bitset<N>`	lights-out puzzles, parity constraints — O(n³/64)

Determinant = product of pivots × (−1)^(#swaps). Rank-nullity counts solutions.

XOR basis = Gaussian elimination on bits

Keep `b[i]` with highest bit i. Insert x: for i from high to low, if bit i set, either store x at b[i] or `x ^= b[i]`.

Invariants (proof: elimination is invertible row operations):

- span(b) = span of all inserted numbers
- rank r ⇒ exactly 2^r distinct subset xors, each hit by 2^(n−r) subsets

Queries:

- **max xor**: greedy from high bit, `r = max(r, r ^ b[i])`
- **contains x**: reduce x; zero at end ⇔ in span
- **k-th smallest**: reduce to RREF, binary digits of k pick basis vectors

XOR basis code

```
struct XorBasis {
    static const int B = 60;
    long long b[B] = {};
    int rank = 0;
    bool insert(long long x) {
        for (int i = B - 1; i >= 0; i--) {
            if (!(x >> i & 1)) continue;
            if (!b[i]) { b[i] = x; rank++; return true; }
            x ^= b[i];
        }
        return false;
    }
    long long maxXor(long long start = 0) const {
        long long r = start;
        for (int i = B - 1; i >= 0; i--) r = max(r, r ^ b[i]);
        return r;
    }
};
```

XOR basis patterns in contests

- **Xor paths in a graph** (CF 845G): any path xor = one DFS path xor ⊕ some subset of fundamental cycle xors → basis of cycles, then `maxXor` /min-xor
- **Range basis** (CF 1100F): for prefix r keep basis that prefers the **latest index** per bit (swap when inserting a newer vector); query uses only vectors with index ≥ l. O((n + q) log C)
- **Counting subsets with xor = x** (CF 959F): 2^(i – rank) if x in span of first i
- **Max number of segments with no zero-xor subset** (CF 1101G): rank of prefix xors
- **Square-free products** (CF 895C): exponent parity vectors over GF(2)

Complexity summary

Tool	Time
k×k matrix power	$O(k^3 \log n)$
Vector × precomputed powers	$O(k^2 \log n)$ per query
FFT / NTT	$O(N \log N)$
Gauss n×m	$O(n\, m \min(n, m))$
Gauss over GF(2) with bitset	$O(n^2\, m / 64)$
XOR basis insert/query	$O(\log C)$

Common pitfalls

- Matrix entries multiplied as `int` → overflow; use `long long` and `% MOD` per term
- Exponents mod **p − 1**, not p (Fermat), and careful when the base ≡ 0
- NTT size must be a power of two ≥ n + m − 1 and divide p − 1
- FFT result rounding: `llround`, check magnitude bound before trusting doubles
- Gauss over reals without partial pivoting → garbage on near-singular systems
- XOR basis with `int` and 1 << 40 → use `1LL` and 64-bit everywhere

Practice problems

- CSES 1722 *Fibonacci Numbers*, 1096 *Throwing Dice*, 1723/1724 *Graph Paths I/II*
- Codeforces 1182E *Product Oriented Recurrence*, 222E *Decoding Genome*
- AtCoder ATC001 C *高速フーリエ変換* (FFT), Library Checker *Convolution*
- Codeforces 528D *Fuzzy Search*, 993E *Nikita and Order Statistics*, 1251F *Red-White Fence*
- Codeforces 24D *Broken robot*, 832E *Vasya and Shifts*, 1155E *Guess the Root*
- SPOJ XMAX, Codeforces 845G, 1100F, 959F, 1101G, 895C

Summary

- Linear processes over n steps ⇒ matrix powers (any semiring)
- Convolutions ⇒ FFT/NTT in O(N log N); learn to *spot* them
- Linear constraints ⇒ Gaussian elimination; choose the field deliberately
- Xor problems ⇒ basis over GF(2); greedy works because of echelon form
- Next lecture: Geometry II